

1. // Selection sort in C

```
#include <stdio.h>

// function to swap the the position of two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void selectionSort(int array[], int size) {
    for (int step = 0; step < size - 1; step++) {
        int min_idx = step;
        for (int i = step + 1; i < size; i++) {

            // To sort in descending order, change > to < in this line.
            // Select the minimum element in each loop.
            if (array[i] < array[min_idx])
                min_idx = i;
        }

        // put min at the correct position
        swap(&array[min_idx], &array[step]);
    }
}

// function to print an array
void printArray(int array[], int size) {
    for (int i = 0; i < size; ++i) {
        printf("%d ", array[i]);
    }
}
```

```

    }

    printf("\n");
}

// driver code
int main() {
    int data[] = {20, 12, 10, 15, 2};
    int size = sizeof(data) / sizeof(data[0]);
    selectionSort(data, size);
    printf("Sorted array in Ascending Order:\n");
    printArray(data, size);
}

```

Output

Sorted array in Ascending Order:

2 10 12 15 20

2. Heap Sort

// Heap Sort in C

```
#include <stdio.h>
```

```
// Function to swap the the position of two elements
```

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

```

```
void heapify(int arr[], int n, int i) {
```

```
    // Find largest among root, left child and right child
```

```

int largest = i;

int left = 2 * i + 1;

int right = 2 * i + 2;

if (left < n && arr[left] > arr[largest])
    largest = left;

if (right < n && arr[right] > arr[largest])
    largest = right;

// Swap and continue heapifying if root is not largest
if (largest != i) {
    swap(&arr[i], &arr[largest]);
    heapify(arr, n, largest);
}
}

// Main function to do heap sort
void heapSort(int arr[], int n) {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Heap sort
    for (int i = n - 1; i >= 0; i--) {
        swap(&arr[0], &arr[i]);

        // Heapify root element to get highest element at root again
        heapify(arr, i, 0);
    }
}

```

```
// Print an array

void printArray(int arr[], int n) {
    for (int i = 0; i < n; ++i)
        printf("%d ", arr[i]);
    printf("\n");
}

// Driver code

int main() {
    int arr[] = {1, 12, 9, 5, 6, 10};
    int n = sizeof(arr) / sizeof(arr[0]);

    heapSort(arr, n);

    printf("Sorted array is \n");
    printArray(arr, n);
}
```

Output

Sorted array is

1 5 6 9 10 12